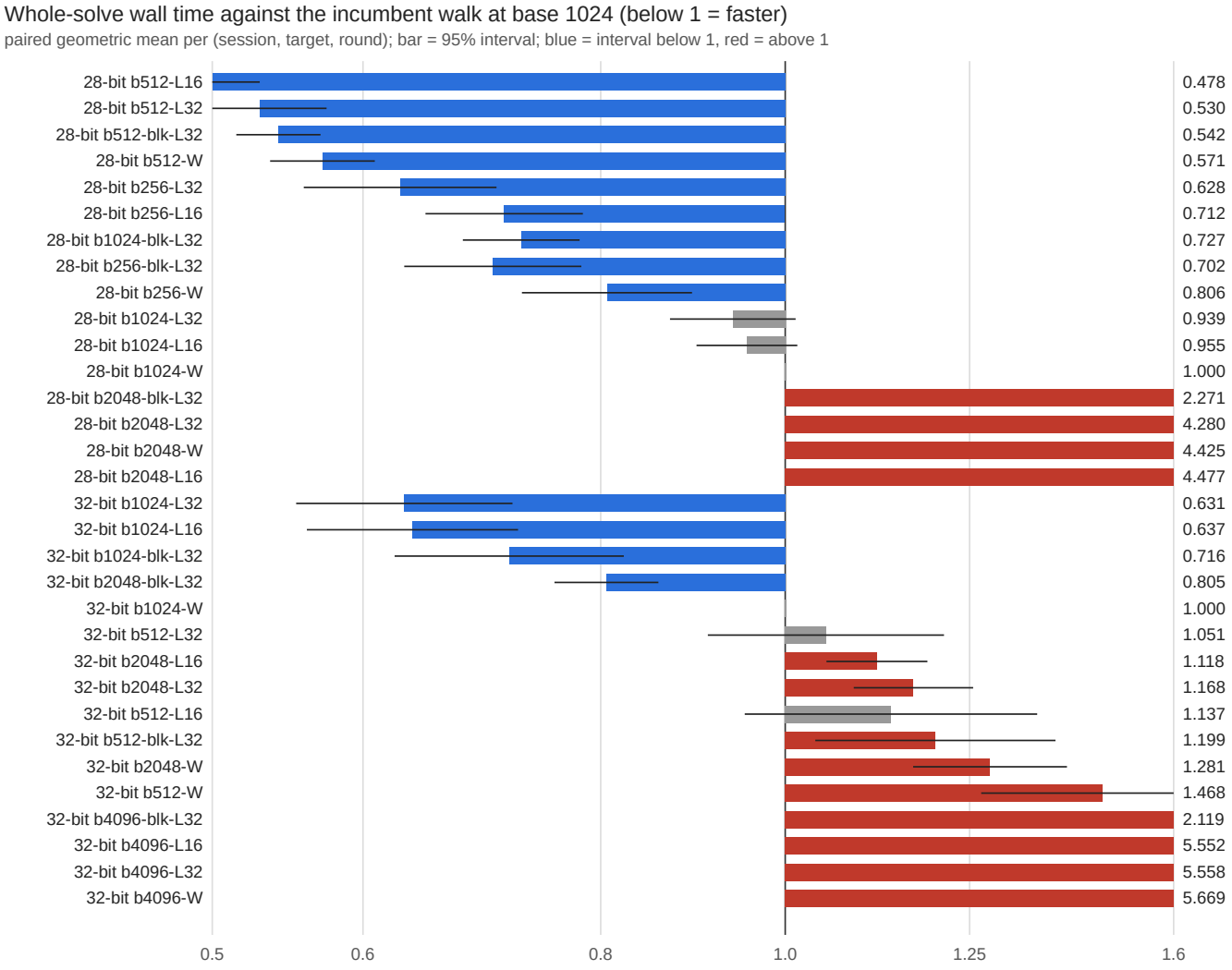
Prime IC relation phase: why wide batching stops paying, and the joint tuning (2026-10-28)

Extension of lead `ICP-prime-ic-constant-engineering` (PR 2783): a diagnosis of the lane-round cost, a check that the code changes move no counted quantity, the `walk-prime-fast` fix, and a joint tuning grid over base size, lane count and oracle.

Findings

- **Where a relation-phase trial goes (28-bit, ns/trial, 16 lanes):** shared-inversion additions 45.9, touch 32.9, lane preparation 18.0, queue 4.9, and 157.1 in the loop body (target draw 19.9, guard insert 55.1, oracle probe 82.0; cycles at 2.1 GHz) of 330.0 in all. The plain walk is 370.4.
- **The shared inversion is amortised by about 32 lanes.** The additions cost 77.7 ns/trial at 4 lanes, 45.9 at 16, 39.6 at 32 and stay at 43.6-46.6 from 256 lanes up (32-bit: 81.3, 51.9, 40.5, 47.9): more lanes buy nothing once the one inversion per round is spread thin.
- **Two costs grow with the width.** (1) *Lane start-up*: each lane begins with two scalar multiplications and an addition, about 12 microseconds a lane, paid once. 1024 lanes cost about 12 ms, a quarter of a 28-bit relation phase and under 5% of a 32-bit one (lane preparation 109.7 ns/trial at 28 bits, 35.1 at 32). This is the same cost the counted S charges, and it is why wide batching loses on small curves and why the knee moves up with the size. (2) *Cache reach*: the round's touched lines (guard slot and table slot, 2 x 64 bytes a lane) must stay in L1 until their trial; at 1024 lanes they do not, and the guard insert and the oracle probe slow down (49.2 + 90.6 ns at 32 lanes, 62.7 + 126.3 at 1024 lanes, 32-bit). Neither can beat the knee, because neither is present at 32 lanes. A smaller lane layout was therefore not built: it could only flatten the tail.
- **The floor is hash-table work.** The guard insert and the oracle probe are about 137.2 ns/trial at 28 bits (139.8 at 32 bits) at the knee, a third to a half of the trial, and do not depend on the width; lanes cannot touch them.
- **Intermediate widths:** relation ns/trial at 28 bits: 16 lanes 330.0, 24 319.3, 32 318.1, 48 327.0, 64 364.1: a flat optimum over 16-48. In the grid, 32 lanes against 16 at 32 bits is 0.991 of the solve time (interval spans 1).
- **`walk-prime-fast` was slower because of its inverse, and is fixed.** It ran at 1.658x the plain walk's relation time (interval 1.58-1.74) because `Fp64::inv` used the binary extended Euclid on 64-bit words; the plain walk's step uses 32-bit word Euclid for moduli below 2^32. A 32-bit word-Euclid path for moduli below 2^32 brings it to 1.042 (interval 0.996-1.090) of the walk, a relation-phase new/old of 0.625 (interval 0.594-0.659). 240 of 240 records are identical outside timing, so no counted quantity moved; the walk and the lane arms are unchanged within noise (new/old 0.995, 0.985, 0.978).
- **Joint tuning: the optimal base does not move.** Whole-solve wall time is lowest at base 512 at 28 bits and 1024 at 32 bits for every method; the build of the B^2/4-entry table grows faster than the relation phase shrinks. Best configurations against the incumbent walk at the same base: 28 bits 51.4 ms (16 lanes) against 62.5 ms; 32 bits 332 ms (32 lanes) against 533 ms. The cache-blocked table (mitm-prime-blocked) halves the build at 2048 and above but its probe is slower, so it only wins at base 4096 (32-bit: 1,092 ms against 2,908 ms).
- **Not done.** The harness's `prime_search` curves stop at 32 bits, so no 34- or 36-bit ladder; chunked inversion and software prefetch have no handle (the table is a std hash map whose slot addresses are not exposed, and the touch loop is already the overlapped-load form); an adaptive width has no room above a flat 16-48 optimum.



28-bit grid

arm (base-targets)	verified	median solve ms	set-up ms	relations ms	mean S	solve vs b1024 walk
b512-L16	24/24	51.4	13.9	34.4	24.9	0.478
b512-L32	24/24	54.5	13.8	39.1	25.5	0.530
b512-blk-L32	24/24	57.9	12.7	43.6	25.5	0.542
b512-W	24/24	62.5	13.4	46.7	24.8	0.571
b256-L32	24/24	73.4	3.3	69.2	20.5	0.628
b256-L16	24/24	76.2	3.3	71.2	21.5	0.712
b1024-blk-L32	24/24	76.2	50.9	21.1	69.2	0.727
b256-blk-L32	24/24	80.1	3.0	76.2	20.5	0.702
b256-W	24/24	86.9	3.3	82.2	19.1	0.806
b1024-L32	24/24	95.7	73.2	17.8	69.2	0.939
b1024-L16	24/24	102.1	77.9	18.4	69.2	0.955
b1024-W	24/24	105.8	76.4	24.1	69.0	1.000
b2048-blk-L32	24/24	238.7	210.6	13.2	262.2	2.271
b2048-L32	24/24	443.6	418.4	10.9	262.2	4.280
b2048-W	24/24	464.2	421.4	17.1	262.3	4.425
b2048-L16	24/24	482.6	442.4	11.4	262.4	4.477

32-bit grid

arm (base-targets)	verified	median solve ms	set-up ms	relations ms	mean S	solve vs b1024 walk
b1024-L32	24/24	332.0	74.6	253.7	31.7	0.631
b1024-L16	24/24	335.0	76.1	247.7	31.1	0.637
b1024-blk-L32	24/24	397.4	51.6	340.2	31.7	0.716
b2048-blk-L32	24/24	414.8	210.6	180.3	78.7	0.805

b1024-W	24/24	533.2	80.5	439.9	33.0	1.000
b512-L32	24/24	580.7	13.7	561.4	33.0	1.051
b2048-L16	24/24	592.1	437.2	135.4	78.0	1.118
b2048-L32	24/24	614.5	422.3	135.6	78.7	1.168
b512-L16	24/24	655.4	14.4	633.7	34.5	1.137
b512-blk-L32	24/24	658.4	12.9	641.2	33.0	1.199
b2048-W	24/24	693.9	434.9	215.5	77.6	1.281
b512-W	24/24	933.2	14.2	911.8	33.0	1.468
b4096-blk-L32	24/24	1091.8	904.7	96.6	288.5	2.119
b4096-L16	24/24	2890.7	2750.8	76.9	288.3	5.552
b4096-L32	24/24	2907.8	2772.4	72.9	288.5	5.558
b4096-W	24/24	2943.7	2696.3	132.4	288.8	5.669